Bank Loan Portfolio & Credit Risk — Python Analysis

This Markdown guide mirrors the Jupyter notebook cell-by-cell. Each analysis step explains **what we are doing, why we are doing it, and which problem-statement requirement it supports**.

Bank Loan Portfolio & Credit Risk — Python Analysis

Objective

This notebook analyzes the raw bank-loan dataset in Python and mirrors the business requirements from the Bank Loan Report problem statement.

The analysis covers:

- Portfolio KPIs: applications, funded amount, repayments, interest rate and DTI
- Good vs Bad Loan performance
- Loan-status analysis
- Monthly lending activity
- Regional/state analysis
- Loan-term analysis
- Home-ownership analysis
- Loan-purpose analysis

- Risk segmentation by grade
- Data-quality validation
- Business insights for lending decisions

> **Important:** The notebook calculates results directly from the raw file. It does not hard-code KPI values. Put your raw `Financial_loan_data.xlsx` file in the same folder as this notebook, or change `DATA_PATH` in the loading cell.

1. Import the Python libraries

What are we doing?

We load the libraries required for data analysis and visualization.

- `pandas` → reading and manipulating tabular loan data
- `numpy` → numerical calculations
- `matplotlib` → general plotting
- `seaborn` → statistical charts
- `plotly` → interactive treemap and U.S. state map

This is the foundation of the analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from pathlib import Path

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.2f

sns.set_theme(style="whitegrid")

print("Libraries imported successfully.")
```

2. Define the raw-data location

What are we doing?

We keep the file location in one variable so the rest of the notebook does not need to repeat the path.

If your raw file has a different name or location, change only this cell.

The expected raw file is `Financial_loan_data.xlsx`.

```
DATA_PATH = Path("Financial_loan_data.xlsx")

if not DATA_PATH.exists():
    # Optional fallback if the raw file is CSV
    csv_path = Path("Financial_loan_data.csv")
    if csv_path.exists():
        DATA_PATH = csv_path

print("Data file:", DATA_PATH)
print("Exists:", DATA_PATH.exists())

if not DATA_PATH.exists():
    raise FileNotFoundError(
        "Raw data file was not found. Put Financial_loan_
        "or Financial_loan_data.csv beside this notebook,
    )
```

3. Load the raw dataset

What are we doing?

We load the original data before creating any analysis columns.

Keeping the raw dataset separate is important because it lets us audit every transformation later.

```
if DATA_PATH.suffix.lower() == ".xlsx":
    df_raw = pd.read_excel(DATA_PATH)
else:
    df_raw = pd.read_csv(DATA_PATH)

df = df_raw.copy()

print(f"Dataset loaded: {df.shape[0]:,} rows × {df.shape[
df.head()
```

4. Inspect the first records

What are we checking?

`head()` gives us a quick view of the raw records.

At this stage we are not changing anything. We are simply understanding how one loan application is represented in the dataset.

```
display(df.head(10))
```

5. Inspect the last records

Why?

Checking the end of the dataset helps confirm that the file loaded completely and that the last records have the expected structure.

```
display(df.tail(10))
```

6. Understand dataset size

What are we measuring?

We identify:

- Number of loan applications
- Number of raw columns

This gives the scale of the portfolio dataset before analysis.

```
rows, cols = df.shape

print("Number of loan applications:", f"{rows:,}")
print("Number of raw columns:", cols)
```

7. Review column names

What are we doing?

We list every raw field so we can map the data to the business questions in the problem statement.

Important fields include `ISSUE_DATE`, `LOAN_STATUS`, `GRADE`, `PURPOSE`, `ADDRESS_STATE`, `TERM`, `ANNUAL_INCOME`, `DTI`, `INT_RATE`, `LOAN_AMOUNT` and `TOTAL_PAYMENT`.

```
pd.DataFrame({"Column": df.columns})
```

8. Check data types and non-null counts

What are we checking?

This tells us whether dates, numbers and categorical fields are stored correctly.

For example:

- Dates should be date/datetime values.
- Loan amount and payment should be numeric.
- Grade, purpose and loan status should be categorical/text values.

```
df.info()
```

9. Statistical overview of numeric fields

What are we doing?

`describe()` provides count, mean, standard deviation, minimum, quartiles and maximum.

This helps us understand the overall financial profile of the loan portfolio and identify suspicious extreme values.

```
numeric_cols = df.select_dtypes(include=np.number).column
display(df[numeric_cols].describe().T)
```

10. Check missing values

Why is this important?

Missing values can affect analysis.

We calculate both the number and percentage of missing records for every field. We do not delete records automatically; first we understand where the missingness occurs and whether the field is important for the required KPIs.

```
missing_summary = (
    pd.DataFrame({
        "Missing_Count": df.isna().sum(),
        "Missing_Percentage": (df.isna().mean() * 100).ro
```

```
    })
    .sort_values("Missing_Count", ascending=False)
)

display(missing_summary[missing_summary["Missing_Count"]
```

11. Visualize missing values

What are we doing?

A bar chart makes it easier to see whether missingness is concentrated in a small number of columns or spread throughout the dataset.

```
missing_plot = missing_summary[missing_summary["Missing_C
missing_plot.columns = ["Column", "Missing_Count", "Missi

if not missing_plot.empty:
    plt.figure(figsize=(10, 5))
    sns.barplot(data=missing_plot, x="Missing_Percentage"
    plt.title("Missing Values by Column")
    plt.xlabel("Missing (%)")
    plt.ylabel("")
    plt.tight_layout()
    plt.show()
else:
    print("No missing values found.")
```

12. Check duplicate records and duplicate loan IDs

Why?

The business metric "Total Loan Applications" assumes each loan application is represented correctly.

We therefore check:

- Entire duplicate rows
- Duplicate `ID`
- Duplicate `MEMBER_ID`

We do not remove duplicates automatically. First we quantify them.

```
duplicate_report = pd.Series({
    "Duplicate full rows": int(df.duplicated().sum()),
    "Duplicate loan IDs": int(df["ID"].duplicated().sum()
    "Duplicate member IDs": int(df["MEMBER_ID"].duplicate
})

display(duplicate_report.to_frame("Count"))
```

13. Validate business-critical numeric fields

What are we checking?

Loan amount, DTI and interest rate should not contain negative values.

This is a business validation check rather than just a technical check.

```
checks = {
    "Negative loan amounts": int((df["LOAN_AMOUNT"] < 0).
    "Negative DTI": int((df["DTI"] < 0).sum()),
    "Negative interest rates": int((df["INT_RATE"] < 0).s
    "Zero loan amounts": int((df["LOAN_AMOUNT"] == 0).sum
}

display(pd.Series(checks, name="Count").to_frame())
```

14. Standardize date fields

What are we doing?

The problem statement requires monthly trend analysis.

We explicitly convert date fields to datetime so that month/year calculations are reliable.

`errors="coerce"` turns invalid dates into missing values instead of crashing the entire workflow.

```
date_cols = [
    "ISSUE_DATE",
    "LAST_CREDIT_PULL_DATE",
    "LAST_PAYMENT_DATE",
    "NEXT_PAYMENT_DATE"
]

for col in date_cols:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce"

print(df[date_cols].dtypes)
```

15. Handle missing employment titles

What are we doing?

If `EMP_TITLE` has missing values, we label them `Unknown`.

We retain the loan applications because employment title is not required for the core portfolio KPIs such as funded amount, repayment, interest rate or DTI.

This avoids unnecessary loss of valid loan records.

```
if "EMP_TITLE" in df.columns:
    df["EMP_TITLE"] = df["EMP_TITLE"].fillna("Unknown")
```

```
print("Remaining missing EMP_TITLE values:",
      int(df["EMP_TITLE"].isna().sum()) if "EMP_TITLE" in
```

Business Analysis

16. Create Good Loan / Bad Loan categories

Business definition

For this project:

- **Good Loan** = `Fully Paid` or `Current`
- **Bad Loan** = `Charged Off`

This classification is used consistently throughout the portfolio-risk analysis.

```
good_statuses = ["Fully Paid", "Current"]

df["LOAN_CATEGORY"] = np.where(
    df["LOAN_STATUS"].eq("Charged Off"),
    "Bad Loan",
    np.where(df["LOAN_STATUS"].isin(good_statuses), "Good
)

display(df["LOAN_CATEGORY"].value_counts(dropna=False).to
```

17. KPI — Total Loan Applications

Business question: How many loan applications are in the portfolio?

We count the loan application ID. If the ID is unique, this represents the total number of applications.

```
total_applications = df["ID"].nunique()
print(f"Total Loan Applications: {total_applications:,}")
```

18. KPI — Total Funded Amount

Business question: How much money did the bank fund?

We sum `LOAN_AMOUNT`.

This represents the amount disbursed/approved across the analyzed loan applications.

```
total_funded = df["LOAN_AMOUNT"].sum()
print(f"Total Funded Amount: ${total_funded:,.0f}")
```

19. KPI — Total Amount Received

Business question: How much has been received from borrowers?

We sum `TOTAL_PAYMENT`.

This is an important repayment and cash-flow indicator for the portfolio.

```
total_received = df["TOTAL_PAYMENT"].sum()
print(f"Total Amount Received: ${total_received:,.0f}")
```

20. KPI — Average Interest Rate

Business question: What is the average interest rate charged across

the portfolio?

The raw dataset stores `INT_RATE` as a decimal, so we multiply the average by 100 to display a percentage.

```
avg_interest_rate = df["INT_RATE"].mean() * 100
print(f"Average Interest Rate: {avg_interest_rate:.2f}%")
```

21. KPI — Average DTI

Business question: What is the average borrower debt-to-income ratio?

`DTI` is stored as a decimal, so the average is multiplied by 100 for percentage presentation.

```
avg_dti = df["DTI"].mean() * 100
print(f"Average DTI: {avg_dti:.2f}%")
```

22. Create a single KPI table

Why?

A single table makes it easy to validate the numbers that should correspond to the Power BI Summary dashboard.

```
kpi_table = pd.DataFrame({
    "KPI": [
        "Total Loan Applications",
        "Total Funded Amount",
        "Total Amount Received",
        "Average Interest Rate",
        "Average DTI"
    ],
```

```
    "Value": [
        f"{total_applications:,}",
        f"${total_funded:,.0f}",
        f"${total_received:,.0f}",
        f"{avg_interest_rate:.2f}%",
        f"{avg_dti:.2f}%"
    ]
})

display(kpi_table)
```

Good vs Bad Loan Analysis

23. Good Loan vs Bad Loan application counts

Business question: What portion of applications are Good Loans versus Bad Loans?

This directly supports the Summary dashboard's Good Loan and Bad Loan visuals.

```
loan_category_counts = (
    df["LOAN_CATEGORY"]
    .value_counts()
    .reindex(["Good Loan", "Bad Loan", "Other"])
    .dropna()
)

display(loan_category_counts.to_frame("Applications"))
```

24. Visualization — Good vs Bad Loan Applications

Why this chart?

A donut chart clearly communicates the composition of the portfolio by loan quality.

This corresponds to the problem statement's Good Loan and Bad Loan donut visuals.

```
plot_data = loan_category_counts[loan_category_counts.ind

plt.figure(figsize=(7, 7))
plt.pie(
    plot_data.values,
    labels=plot_data.index,
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"width": 0.45}
)
plt.title("Good vs Bad Loan Applications")
plt.tight_layout()
plt.show()
```

25. Good vs Bad Loan percentage

What are we calculating?

We calculate the percentage of total applications represented by Good Loans and Bad Loans.

This is useful for risk monitoring because a rising Bad Loan percentage can indicate portfolio deterioration.

```
loan_quality_summary = (
```

```
    df.groupby("LOAN_CATEGORY")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Amount_Received=("TOTAL_PAYMENT", "sum")
      )
)

loan_quality_summary["Application_Pct"] = (
    loan_quality_summary["Applications"] / len(df) * 100
).round(2)

display(loan_quality_summary)
```

Loan Status Analysis

26. Loan-status distribution

Business question: What is the current status of the loan portfolio?

This supports the "Loan Status wise Loan Applications" visual from the problem statement.

```
status_summary = (
    df.groupby("LOAN_STATUS")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Amount_Received=("TOTAL_PAYMENT", "sum")
      )
      .sort_values("Applications", ascending=False)
)
```

```
display(status_summary)
```

27. Visualization — Applications by Loan Status

Why?

A bar chart makes the volume of each status easy to compare.

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=status_summary.reset_index(),
    x="LOAN_STATUS",
    y="Applications"
)
plt.title("Loan Applications by Loan Status")
plt.xlabel("Loan Status")
plt.ylabel("Applications")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

28. Visualization — Funded Amount vs Amount Received by Loan Status

Business question: How much was funded versus how much was received for each loan status?

This directly supports the Summary dashboard requirement.

```
status_money = status_summary.reset_index().melt(
    id_vars="LOAN_STATUS",
    value_vars=["Funded_Amount", "Amount_Received"],
```

```
    var_name="Metric",
    value_name="Amount"
)

plt.figure(figsize=(10, 6))
sns.barplot(
    data=status_money,
    x="LOAN_STATUS",
    y="Amount",
    hue="Metric"
)
plt.title("Funded Amount vs Amount Received by Loan Statu
plt.xlabel("Loan Status")
plt.ylabel("Amount")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

Risk Analysis

29. Average Interest Rate by Loan Status

Business question: Do interest rates differ across loan statuses?

This helps connect pricing with portfolio performance and supports the required Summary visual.

```
interest_by_status = (
    df.groupby("LOAN_STATUS", as_index=False)["INT_RATE"]
      .mean()
)

interest_by_status["Average_Interest_Rate_Pct"] = (
```

```
    interest_by_status["INT_RATE"] * 100
)

display(interest_by_status[["LOAN_STATUS", "Average_Inter
```

30. Visualization — Average Interest Rate by Loan Status

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=interest_by_status,
    x="LOAN_STATUS",
    y="Average_Interest_Rate_Pct"
)
plt.title("Average Interest Rate by Loan Status")
plt.xlabel("Loan Status")
plt.ylabel("Average Interest Rate (%)")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

31. Average DTI by Loan Status

Business question: Is borrower leverage different across loan statuses?

DTI is a useful borrower financial-health indicator and is explicitly required in the problem statement.

```
dti_by_status = (
    df.groupby("LOAN_STATUS", as_index=False)["DTI"]
      .mean()
)
```

```
dti_by_status["Average_DTI_Pct"] = dti_by_status["DTI"] *

display(dti_by_status[["LOAN_STATUS", "Average_DTI_Pct"]]
```

32. Visualization — Average DTI by Loan Status

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=dti_by_status,
    x="LOAN_STATUS",
    y="Average_DTI_Pct"
)
plt.title("Average DTI by Loan Status")
plt.xlabel("Loan Status")
plt.ylabel("Average DTI (%)")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

33. Bad Loan Rate by Grade

Business question: Which credit grades carry greater portfolio risk?

Grade is the bank's overall risk classification. We calculate the percentage of loans charged off within each grade.

This adds a credit-risk perspective to the dashboard analysis.

```
grade_risk = (
    df.groupby("GRADE")
      .agg(
          Applications=("ID", "count"),
```

```
            Bad_Loans=("LOAN_CATEGORY", lambda x: (x == "Ba
            Funded_Amount=("LOAN_AMOUNT", "sum")
        )
        .reset_index()
)

grade_risk["Bad_Loan_Rate_Pct"] = (
    grade_risk["Bad_Loans"] / grade_risk["Applications"]
).round(2)

grade_risk = grade_risk.sort_values("Bad_Loan_Rate_Pct",

display(grade_risk)
```

34. Visualization — Bad Loan Rate by Grade

Why?

A grade-level risk chart highlights where credit quality is deteriorating and helps support lending-policy decisions.

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=grade_risk.sort_values("GRADE"),
    x="GRADE",
    y="Bad_Loan_Rate_Pct"
)
plt.title("Bad Loan Rate by Credit Grade")
plt.xlabel("Grade")
plt.ylabel("Bad Loan Rate (%)")
plt.tight_layout()
plt.show()
```

Dashboard 2 — Overview

Analysis

35. Monthly Loan Applications

Business question: How did lending activity change over time?

We extract the month from `ISSUE_DATE` and count applications.

This directly corresponds to the required monthly loan-application line chart.

```
monthly = (
    df.assign(Month=df["ISSUE_DATE"].dt.to_period("M"))
      .groupby("Month")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Amount_Received=("TOTAL_PAYMENT", "sum")
      )
      .reset_index()
)

monthly["Month"] = monthly["Month"].dt.to_timestamp()

display(monthly)
```

36. Visualization — Monthly Loan Applications

Why?

A line chart reveals growth, decline and seasonal changes in loan

demand.

```
plt.figure(figsize=(12, 5))
sns.lineplot(
    data=monthly,
    x="Month",
    y="Applications",
    marker="o"
)
plt.title("Monthly Loan Applications")
plt.xlabel("Issue Month")
plt.ylabel("Applications")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

37. Monthly funded amount trend

Why add this?

Applications tell us volume, but funded amount tells us the monetary scale of lending.

This provides additional portfolio context while remaining consistent with the lending-performance objective.

```
plt.figure(figsize=(12, 5))
sns.lineplot(
    data=monthly,
    x="Month",
    y="Funded_Amount",
    marker="o"
)
plt.title("Monthly Funded Amount")
plt.xlabel("Issue Month")
plt.ylabel("Funded Amount")
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```

38. Loan Term Analysis

Business question: What repayment terms are most common?

The problem statement specifically requests a donut chart for loan-term analysis.

```
term_summary = (
    df.assign(TERM_CLEAN=df["TERM"].astype(str).str.strip
      .groupby("TERM_CLEAN")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum")
      )
      .reset_index()
      .sort_values("Applications", ascending=False)
)

display(term_summary)
```

39. Visualization — Loan Term Donut

```
plt.figure(figsize=(7, 7))
plt.pie(
    term_summary["Applications"],
    labels=term_summary["TERM_CLEAN"],
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"width": 0.45}
)
plt.title("Loan Term Analysis")
```

```
plt.tight_layout()
plt.show()
```

40. Home Ownership Analysis

Business question: How are borrowers distributed by home ownership?

This corresponds to the required Home Ownership bar chart.

```
home_summary = (
    df.groupby("HOME_OWNERSHIP")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Bad_Loans=("LOAN_CATEGORY", lambda x: (x == "Ba
      )
      .reset_index()
)

home_summary["Bad_Loan_Rate_Pct"] = (
    home_summary["Bad_Loans"] / home_summary["Application
).round(2)

display(home_summary.sort_values("Applications", ascendin
```

41. Visualization — Home Ownership

```
plt.figure(figsize=(9, 5))
sns.barplot(
    data=home_summary.sort_values("Applications", ascendi
    x="HOME_OWNERSHIP",
    y="Applications"
)
```

```
plt.title("Loan Applications by Home Ownership")
plt.xlabel("Home Ownership")
plt.ylabel("Applications")
plt.tight_layout()
plt.show()
```

42. Loan Purpose Breakdown

Business question: Why are customers borrowing?

The problem statement requires a treemap for loan-purpose analysis.

```
purpose_summary = (
    df.groupby("PURPOSE")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Amount_Received=("TOTAL_PAYMENT", "sum")
      )
      .reset_index()
      .sort_values("Applications", ascending=False)
)

display(purpose_summary)
```

43. Visualization — Loan Purpose Treemap

Why a treemap?

A treemap shows both the number and relative importance of purposes in a compact visual.

The rectangle size represents applications.

```
fig = px.treemap(
    purpose_summary,
    path=["PURPOSE"],
    values="Applications",
    title="Loan Purpose Breakdown"
)

fig.show()
```

44. Regional Analysis by State

Business question: Where is the loan portfolio concentrated?

We aggregate applications and funded amount by borrower state.

This creates the data required for the regional analysis visual.

```
state_summary = (
    df.groupby("ADDRESS_STATE")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Bad_Loans=("LOAN_CATEGORY", lambda x: (x == "Ba
      )
      .reset_index()
)

state_summary["Bad_Loan_Rate_Pct"] = (
    state_summary["Bad_Loans"] / state_summary["Applicati
).round(2)

display(state_summary.sort_values("Applications", ascendi
```

45. Visualization — Regional Analysis Map

Why?

The map shows the geographic distribution of applications across U.S. states.

This corresponds to the Filled Map requirement in the problem statement.

```
fig = px.choropleth(
    state_summary,
    locations="ADDRESS_STATE",
    locationmode="USA-states",
    color="Applications",
    scope="usa",
    hover_name="ADDRESS_STATE",
    hover_data={
        "Applications": True,
        "Funded_Amount": ":,.0f",
        "Bad_Loan_Rate_Pct": ":.2f"
    },
    title="Regional Loan Applications by State"
)

fig.show()
```

46. Top states by funded amount

Why?

The map shows geographic distribution, while this table makes the highest-funded markets immediately identifiable.

```
top_states = (
    state_summary
    .sort_values("Funded_Amount", ascending=False)
    .head(10)
)
```

```
display(top_states)
```

Additional Portfolio Insights

47. Purpose-level risk analysis

Business question: Which borrowing purposes have the highest Bad Loan rate?

This combines business purpose with credit-risk performance.

```
purpose_risk = (
    df.groupby("PURPOSE")
      .agg(
          Applications=("ID", "count"),
          Bad_Loans=("LOAN_CATEGORY", lambda x: (x == "Ba
          Funded_Amount=("LOAN_AMOUNT", "sum")
      )
      .reset_index()
)

purpose_risk["Bad_Loan_Rate_Pct"] = (
    purpose_risk["Bad_Loans"] / purpose_risk["Application
).round(2)

display(
    purpose_risk.sort_values(
        ["Bad_Loan_Rate_Pct", "Applications"],
        ascending=[False, False]
    )
)
```

48. Grade-level portfolio concentration

Why?

We compare the number of applications and funded amount by grade.

This helps distinguish between high-volume grades and high-risk grades.

```
grade_summary = (
    df.groupby("GRADE")
      .agg(
          Applications=("ID", "count"),
          Funded_Amount=("LOAN_AMOUNT", "sum"),
          Amount_Received=("TOTAL_PAYMENT", "sum")
      )
      .reset_index()
)

grade_summary["Application_Share_Pct"] = (
    grade_summary["Applications"] / len(df) * 100
).round(2)

display(grade_summary.sort_values("Applications", ascendi
```

49. Final portfolio summary

What are we doing?

We bring the most important outputs into one table for final validation against the Power BI dashboard.

The purpose is not to duplicate Power BI, but to verify that Python, SQL and Power BI are using consistent business definitions.

```
final_summary = pd.DataFrame({
```

```
    "Metric": [
        "Applications",
        "Funded Amount",
        "Amount Received",
        "Average Interest Rate",
        "Average DTI",
        "Good Loan Applications",
        "Bad Loan Applications"
    ],
    "Value": [
        f"{len(df):,}",
        f"${df['LOAN_AMOUNT'].sum():,.0f}",
        f"${df['TOTAL_PAYMENT'].sum():,.0f}",
        f"{df['INT_RATE'].mean() * 100:.2f}%",
        f"{df['DTI'].mean() * 100:.2f}%",
        f"{(df['LOAN_CATEGORY'] == 'Good Loan').sum():,}"
        f"{(df['LOAN_CATEGORY'] == 'Bad Loan').sum():,}"
    ]
})

display(final_summary)
```

50. Analyst conclusion

After running the notebook, use the calculated outputs to write the final business conclusion.

Focus on:

1. Overall lending volume and funded amount
2. Repayment amount relative to funding
3. Good vs Bad Loan composition
4. Loan-status concentration
5. Interest-rate differences across statuses
6. DTI differences across statuses
7. High-risk grades

8. Monthly lending trend
9. Geographic concentration
10. Major loan purposes
11. Loan-term and home-ownership patterns

Do **not** write conclusions before running the notebook. The raw data should determine the final findings.

Visualization-to-Problem-Statement Mapping

Problem Statement Requirement	Python Output
Total Loan Applications	KPI cells 17–22
Total Funded Amount	KPI cells 18–22
Total Amount Received	KPI cells 19–22
Average Interest Rate	KPI cells 20–22
Average DTI	KPI cells 21–22
Good Loan Applications	Cells 23–25
Bad Loan Applications	Cells 23–25
Funded vs Repayment by Loan Status	Cell 28
Loan Status-wise Applications	Cell 27
Average Interest Rate by Loan Status	Cells 29–30
Average DTI by Loan Status	Cells 31–32
Monthly Loan Applications	Cells 35–36
Regional Analysis	Cells 44–46
Loan Term Analysis	Cells 38–39
Home Ownership Analysis	Cells 40–41

Loan Purpose Breakdown	Cells 42–43
Risk/Grade Analysis	Cells 33–34 and 48

Final validation checklist

Before using the Python results in your project:

- ☐ Raw row count matches the source dataset
- ☐ Duplicate loan IDs are checked
- ☐ Missing values are reviewed
- ☐ Dates are converted correctly
- ☐ `INT_RATE` and `DTI` are displayed as percentages
- ☐ Good Loan = Fully Paid / Current
- ☐ Bad Loan = Charged Off
- ☐ KPI values are compared with SQL
- ☐ KPI values are compared with Power BI
- ☐ Monthly totals reconcile
- ☐ State totals reconcile
- ☐ No conclusions are written from unvalidated numbers